

I need a Python function called "build_activity_vocab(df: pd.DataFrame) → dict" that takes a DataFrame with a column 'prefix_activities' containing lists of activity name strings, collects all unique activity names across all prefixes, and returns a dictionary mapping each activity name to a unique integer index starting from 1. Index 0 is reserved for padding

👉 LISA Flash

Here is the Python function to build the activity vocabulary as requested. It iterates through the lists in the `prefix_activities` column, collects unique names, sorts them (to ensure consistent indexing), and maps them to integers starting from 1.

python

⌵ Свернуть ⌵ Запустить ⌵ Копировать

```
import pandas as pd

def build_activity_vocab(df: pd.DataFrame) -> dict:
    """
    Builds a vocabulary dictionary mapping unique activity names to integer
    indices.

    Args:
        df (pd.DataFrame): DataFrame containing a 'prefix_activities' column with
        lists of strings.

    Returns:
        dict: A dictionary mapping each unique activity name to a unique integer
        index.

        Index 0 is reserved for padding and is not included in this mapping.
    """
    # Collect all unique activity names from the lists in the 'prefix_activities'
    column
    unique_activities = set()

    for activity_list in df['prefix_activities']:
        if isinstance(activity_list, list):
            unique_activities.update(activity_list)

    # Sort the activities to ensure consistent indexing across runs
    sorted_activities = sorted(list(unique_activities))

    # Create the mapping: start index from 1
    vocab = {activity: idx + 1 for idx, activity in enumerate(sorted_activities)}

    return vocab
```

return vocab

Example Usage

python

⌵ Свернуть ⌵ Запустить ⌵ Копировать

```
# Sample data
data = {
    'prefix_activities': [
        ['running', 'swimming'],
        ['cycling', 'running'],
        ['yoga']
    ]
}

df_example = pd.DataFrame(data)

vocab = build_activity_vocab(df_example)
print(vocab)

# Output might look like: {'cycling': 1, 'running': 2, 'swimming': 3, 'yoga': 4}
# Note: The order depends on sorting, but indices will always be 1-based and
unique.
```

I need a Python function called "encode_and_pad(prefixes: list, vocab: dict, max_len: int) → torch.Tensor" that takes a list of activity name lists, a vocabulary dictionary, and a maximum sequence length. Each activity list should be converted to integer indices using the vocab, then padded with zeros on the left to reach max_len. The function should return a 2D torch.Tensor of shape (len(prefixes), max_len) with dtype torch.long

👉 LISA Flash

Here is the Python function `encode_and_pad` that performs the requested encoding and padding logic using PyTorch.

python

⌵ Свернуть ⌵ Запустить ⌵ Копировать

```
import torch
from typing import List, Dict

def encode_and_pad(prefixes: List[List[str]], vocab: Dict[str, int], max_len: int)
    -> torch.Tensor:
    """
    Encodes a list of activity name lists into integer indices and pads them to a
    fixed length.

    Args:
        prefixes (List[List[str]]): A list where each element is a list of activity
        name strings.
```

```

vocab (Dict[str, int]): A dictionary mapping activity names to integer
indices (starting from 1).

max_len (int): The maximum sequence length. Sequences shorter than this
will be padded with 0s on the left.

Returns:
    torch.Tensor: A 2D tensor of shape (len(prefixes), max_len) with dtype
torch.long.

        Padding index 0 is used for left-padding.
"""
encoded_sequences = []

for prefix in prefixes:
    # Convert string activities to their corresponding integer indices
    # If an activity is not in vocab, it defaults to 0 (though typically vocab
should cover all inputs)
    indices = [vocab.get(activity, 0) for activity in prefix]

    # Calculate current length
    current_len = len(indices)

    if current_len > max_len:
        # Truncate if longer than max_len (taking the last max_len elements
usually makes sense for sequences)
        # Alternatively, you might want to take the first max_len.
        # Given "prefix" context, keeping the end or start depends on specific
use case.
        # Standard practice for variable length padding often truncates the
tail or head.
        # Here we truncate the tail to keep the 'prefix' intact if possible,
but strictly speaking
        # if it's a "prefix", maybe we keep the beginning?
        # Let's assume standard truncation to fit max_len (keeping the start is
safer for "prefixes").
        indices = indices[:max_len]
        current_len = max_len

    elif current_len < max_len:
        # Pad with 0s on the LEFT to reach max_len
        padding_needed = max_len - current_len
        indices = [0] * padding_needed + indices

```



```

        indices = [0] * padding_needed + indices

    encoded_sequences.append(indices)

# Convert list of lists to a torch tensor
tensor = torch.tensor(encoded_sequences, dtype=torch.long)

return tensor

```

Example Usage

```

python


⌵ Свернуть



⏏ Запустить



📋 Копировать


```

```

# Setup
vocab = {'running': 1, 'swimming': 2, 'cycling': 3}
prefixes = [
    ['running', 'swimming'],      # Length 2
    ['cycling'],                  # Length 1
    ['running', 'swimming', 'cycling', 'yoga'] # Length 4 (assuming yoga is OOV or
handled)
]
max_len = 4

# Note: If 'yoga' is not in vocab, it becomes 0.
# If you expect all items to be in vocab, ensure vocab is complete.
# Let's add 'yoga' to vocab for this example to avoid OOV zeros in data.
vocab['yoga'] = 4

result = encode_and_pad(prefixes, vocab, max_len)

print(result)
print(f"Shape: {result.shape}")
print(f"Dtype: {result.dtype}")

# Expected Output Logic:
# Row 1: ['running', 'swimming'] -> [1, 2] -> pad left 2 zeros -> [0, 0, 1, 2]
# Row 2: ['cycling'] -> [3] -> pad left 3 zeros -> [0, 0, 0, 3]
# Row 3: ['running', 'swimming', 'cycling', 'yoga'] -> [1, 2, 3, 4] -> no padding
needed -> [1, 2, 3, 4]

```

I need a Python class called "SeldAttentionBlock" that inherits from torch.nn.Module. The constructor takes three arguments: "embed_dim" (integer, embedding dimension), "num_heads" (integer, number of attention heads), "dropout" (float, dropout rate). In the

constructor, create: a torch.nn.MultiheadAttention layer with embed_dim, num_heads, dropout, and batch_first=True; a torch.nn.LayerNorm with embed_dim; a torch.nn.Dropout with the given rate. In the forward method, take input tensor x of shape (batch, seq_len, embed_dim), apply MultiheadAttention with x as query, key and value, apply dropout to the attention output, add a residual connection with the input x, and apply LayerNorm. Return the result

👉 LISA Flash

Here is the Python class `SeldAttentionBlock` implementing the specified Self-Attention logic with residual connections and Layer Normalization.

python

⌵ Свернуть ⌵ Запустить ⌵ Копировать

```
import torch
import torch.nn as nn

class SeldAttentionBlock(nn.Module):
    def __init__(self, embed_dim: int, num_heads: int, dropout: float = 0.0):
        """
        Initializes the SELD Attention Block.

        Args:
            embed_dim (int): Dimensionality of the input embeddings.
            num_heads (int): Number of parallel attention heads.
            dropout (float): Dropout rate applied after the attention mechanism.
        """
        super(SeldAttentionBlock, self).__init__()

        # Multihead Attention layer
        # batch_first=True ensures input shape is (Batch, Seq_Len, Embed_Dim)
        self.attention = nn.MultiheadAttention(
            embed_dim=embed_dim,
            num_heads=num_heads,
            dropout=dropout,
            batch_first=True
        )

        # Layer Normalization
        self.layer_norm = nn.LayerNorm(embed_dim)

        # Dropout layer
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
```

Forward pass for the attention block.

Args:

`x (torch.Tensor)`: Input tensor of shape (batch, seq_len, embed_dim).

Returns:

`torch.Tensor`: Output tensor of shape (batch, seq_len, embed_dim).

"""

Apply MultiheadAttention

query, key, value are all 'x' for self-attention

`attn_output, _ = self.attention(query=x, key=x, value=x)`

Apply dropout to the attention output

`attn_output = self.dropout(attn_output)`

Add residual connection (input x + dropout output)

`residual = x + attn_output`

Apply Layer Normalization

`output = self.layer_norm(residual)`

`return output`

The class name contains a typo: it is named "SeldAttentionBlock" but should be "SelfAttentionBlock". Please provide the corrected class with the right name

👋 LISA Flash

Here is the corrected Python class with the proper name `SelfAttentionBlock`:

python

⌵ Свернуть ⌵ Запустить ⌵ Копировать

```
import torch
```

```
import torch.nn as nn
```

```
class SelfAttentionBlock(nn.Module):
```

```
    def __init__(self, embed_dim: int, num_heads: int, dropout: float = 0.0):
```

```
        """
```

```
        Initializes the Self Attention Block.
```

Args:

`embed_dim (int)`: Dimensionality of the input embeddings.

`num_heads (int)`: Number of parallel attention heads.

`dropout (float)`: Dropout rate applied after the attention mechanism.

dropout (float): Dropout rate applied after the attention mechanism.

```
"""
super(SelfAttentionBlock, self).__init__()

# Multihead Attention layer
# batch_first=True ensures input shape is (Batch, Seq_Len, Embed_Dim)
self.attention = nn.MultiheadAttention(
    embed_dim=embed_dim,
    num_heads=num_heads,
    dropout=dropout,
    batch_first=True
)

# Layer Normalization
self.layer_norm = nn.LayerNorm(embed_dim)

# Dropout layer
self.dropout = nn.Dropout(dropout)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    Forward pass for the attention block.

    Args:
        x (torch.Tensor): Input tensor of shape (batch, seq_len, embed_dim).

    Returns:
        torch.Tensor: Output tensor of shape (batch, seq_len, embed_dim).
    """
    # Apply MultiheadAttention
    # query, key, value are all 'x' for self-attention
    attn_output, _ = self.attention(query=x, key=x, value=x)

    # Apply dropout to the attention output
    attn_output = self.dropout(attn_output)

    # Add residual connection (input x + dropout output)
    residual = x + attn_output

    # Apply Layer Normalization
    output = self.layer_norm(residual)
```

```
return output
```

I need a Python class called "ProcessTransformer" that inherits from "torch.nn.Module". The constructor takes: "vocab_size" (int), "embed_dim" (int, default 64), "num_heads" (int, default 4), "num_layers" (int, default 2), "num_activities" (int, number of unique activities for classification output), "dropout" (float, default 0.1), "max_len" (int, maximum sequence length). In the constructor, create: an Embedding layer of shape (vocab_size+1, embed_dim) with padding_idx=0; a list of "num_layers" SelfAttentionBlock instances wrapped in nn.ModuleList; a Linear layer from embed_dim to num_activities for activity prediction head; a Linear layer from embed_dim to 1 for remaining time prediction head. In the forward method, take input tensor x of shape (batch, seq_len), apply embedding, pass sequentially through all SelfAttentionBlocks, take the mean over the sequence dimension, then pass through both output heads and return a tuple (activity_logits, time_prediction) where activity_logits has shape (batch, num_activities) and time_prediction has shape (batch, 1)

👉 LISA Flash

Here is the `ProcessTransformer` class implementation. It includes the embedding layer, a stack of self-attention blocks, and two distinct output heads for activity classification and time regression.

python

🔗 Свернуть Запустить Копировать

```
import torch
import torch.nn as nn
from typing import Tuple

# Assuming SelfAttentionBlock is defined in the same scope or imported from the
previous step
# If not, ensure the definition provided earlier is available.
```

```
class ProcessTransformer(nn.Module):
    def __init__(
        self,
        vocab_size: int,
        embed_dim: int = 64,
        num_heads: int = 4,
        num_layers: int = 2,
        num_activities: int = 10,
        dropout: float = 0.1,
        max_len: int = 50
    ):
        """
```

Initializes the Process Transformer model.

Args:

vocab_size (int): Size of the vocabulary (excluding padding index).
 embed_dim (int): Dimensionality of the embeddings.
 num_heads (int): Number of attention heads.
 num_layers (int): Number of transformer layers.
 num_activities (int): Number of unique activity classes for

classification.

dropout (float): Dropout rate.

max_len (int): Maximum sequence length (used for positional encoding if added,

though not explicitly requested here, it's stored for context).

"""

super(ProcessTransformer, self).__init__()

self.embed_dim = embed_dim

self.num_activities = num_activities

Embedding layer: (vocab_size + 1) to account for padding index 0

self.embedding = nn.Embedding(
 num_embeddings=vocab_size + 1,
 embedding_dim=embed_dim,
 padding_idx=0
)

Stack of SelfAttentionBlocks

self.layers = nn.ModuleList([
 SelfAttentionBlock(
 embed_dim=embed_dim,
 num_heads=num_heads,
 dropout=dropout
)
 for _ in range(num_layers)
])

Activity prediction head: Linear from embed_dim to num_activities

self.activity_head = nn.Linear(embed_dim, num_activities)

Remaining time prediction head: Linear from embed_dim to 1

```

self.time_head = nn.Linear(embed_dim, 1)

# Store max_len for potential future use (e.g., positional encodings)
self.max_len = max_len

def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
    """
    Forward pass of the model.

    Args:
        x (torch.Tensor): Input tensor of shape (batch, seq_len) containing
token indices.

    Returns:
        Tuple[torch.Tensor, torch.Tensor]:
            - activity_logits: Tensor of shape (batch, num_activities)
            - time_prediction: Tensor of shape (batch, 1)
    """
    batch_size, seq_len = x.shape

    # 1. Apply Embedding
    # Output shape: (batch, seq_len, embed_dim)
    x_embed = self.embedding(x)

    # 2. Pass through SelfAttention Blocks sequentially
    for layer in self.layers:
        x_embed = layer(x_embed)

    # 3. Global Pooling: Mean over the sequence dimension
    # Reduce dim 1 (seq_len) to get a single vector per sample
    # Shape becomes: (batch, embed_dim)
    pooled_output = x_embed.mean(dim=1)

    # 4. Pass through Output Heads
    # Activity Logits
    activity_logits = self.activity_head(pooled_output) # Shape: (batch,
num_activities)

    # Time Prediction
    time_prediction = self.time_head(pooled_output) # Shape: (batch, 1)

```

```
return activity_logits, time_prediction
```

I need a Python function called "train_model(model, train_loader, num_epochs: int, learning_rate: float, device: str) → list" that trains a ProcessTransformer model. The function should: use Adam optimizer with the given learning_rate; use CrossEntropyLoss for the activity prediction head and MSELoss for the remaining time head; combine the two losses by summing them with equal weight; iterate for num_epochs epochs over train_loader where each batch contains (x, y_activity, y_time); move all tensors to the given device; call optimizer.zero_grad(), forward pass, compute combined loss, loss.backward(), optimizer.step(); track and print average combined loss per epoch; return a list of per-epoch average losses.

👉 LISA Flash

Here is the `train_model` function implementing the training loop with the specified loss functions, optimizer, and device handling.

python

⌵ Свернуть ⏏ Запустить 📋 Копировать

```
import torch
import torch.nn as nn
from typing import List, Tuple, Any
```

```
def train_model(
    model: nn.Module,
    train_loader: torch.utils.data.DataLoader,
    num_epochs: int,
    learning_rate: float,
    device: str
) -> List[float]:
    """
    Trains the ProcessTransformer model for a specified number of epochs.
```

Args:

`model` (nn.Module): The ProcessTransformer model to train.
`train_loader` (DataLoader): DataLoader yielding batches of (x, y_activity, y_time).
`num_epochs` (int): Number of training epochs.
`learning_rate` (float): Learning rate for the Adam optimizer.
`device` (str): Device string (e.g., 'cpu' or 'cuda').

Returns:

`List[float]`: A list containing the average combined loss for each epoch.


```
#####  
# Set up Optimizer and Loss Functions  
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)  
  
# Activity loss: CrossEntropyLoss (expects raw logits and class indices)  
criterion_activity = nn.CrossEntropyLoss()  
  
# Time loss: MSELoss  
criterion_time = nn.MSELoss()  
  
# Move model to device  
model.to(device)  
  
epoch_losses = []  
  
for epoch in range(num_epochs):  
    model.train()  
    total_loss = 0.0  
    num_batches = 0  
  
    for batch in train_loader:  
        # Unpack batch: (x, y_activity, y_time)  
        x, y_activity, y_time = batch  
  
        # Move tensors to device  
        x = x.to(device)  
        y_activity = y_activity.to(device)  
        y_time = y_time.to(device)  
  
        # Zero gradients  
        optimizer.zero_grad()  
  
        # Forward pass  
        activity_logits, time_prediction = model(x)  
  
        # Compute losses  
        # activity_logits shape: (batch, num_activities)  
        # y_activity shape: (batch,) - integer class indices  
        loss_activity = criterion_activity(activity_logits, y_activity.long())  
  
        # time prediction shape: (batch, 1)
```

```

        # y_time shape: (batch, 1) or (batch,) depending on loader, ensure
match
        # If y_time is (batch,), unsqueeze to (batch, 1) for MSELoss
consistency if needed
        if y_time.dim() == 1:
            y_time = y_time.unsqueeze(1)

        loss_time = criterion_time(time_prediction, y_time.float())

        # Combined loss (equal weight)
        loss = loss_activity + loss_time

        # Backward pass
        loss.backward()

        # Update weights
        optimizer.step()

        # Accumulate loss
        total_loss += loss.item()
        num_batches += 1

    # Calculate average loss for the epoch
    avg_epoch_loss = total_loss / num_batches
    epoch_losses.append(avg_epoch_loss)

    print(f"Epoch {epoch+1}/{num_epochs} - Average Combined Loss:
{avg_epoch_loss:.4f}")

    return epoch_losses

```

I need a Python function called "create_data_loader(prefixes: pd.DataFrame, encoded_x: torch.Tensor, vocab: dict, batch_size: int = 32, shuffle: bool = True) → torch.utils.data.DataLoader" that creates a DataLoader for training. The function should: create y_activity by mapping the 'next_activity' column of prefixes to integer indices using vocab (unknown activities map to 0); create y_time from the 'remaining_time' column as a float tensor; combine encoded_x, y_activity, y_time into a TensorDataset; return a DataLoader with the given batch_size and shuffle settings

here is the `create_data_loader` function that constructs the necessary tensors and returns a PyTorch

`DataLoader`.

python

⌵ Свернуть ⌵ Запустить ⌵ Копировать

```
import pandas as pd
import torch
from torch.utils.data import TensorDataset, DataLoader
from typing import Dict

def create_data_loader(
    prefixes: pd.DataFrame,
    encoded_x: torch.Tensor,
    vocab: Dict[str, int],
    batch_size: int = 32,
    shuffle: bool = True
) -> DataLoader:
    """
    Creates a DataLoader for training from a DataFrame and encoded tensor.

    Args:
        prefixes (pd.DataFrame): DataFrame containing 'next_activity' and
        'remaining_time' columns.
        encoded_x (torch.Tensor): Encoded input sequences of shape (N, max_len).
        vocab (dict): Vocabulary mapping activity names to indices.
        batch_size (int): Batch size for the DataLoader.
        shuffle (bool): Whether to shuffle the data.

    Returns:
        DataLoader: A PyTorch DataLoader yielding batches of (x, y_activity,
        y_time).
    """
    # 1. Create y_activity (Next Activity Labels)
    # Map 'next_activity' strings to integers using vocab.
    # Unknown activities map to 0 (padding index).
    # Ensure the result is a Long tensor for CrossEntropyLoss.
    next_activities = prefixes['next_activity'].apply(lambda x: vocab.get(x, 0))
    y_activity = torch.tensor(next_activities.values, dtype=torch.long)

    # 2. Create y_time (Remaining Time Labels)
    # Convert 'remaining_time' column to float tensor.
    remaining_times = prefixes['remaining_time']
    y_time = torch.tensor(remaining_times.values, dtype=torch.float32)
```



```

# Ensure y_time has shape (N, 1) if it's currently (N,), to match model output
expectations
if y_time.dim() == 1:
    y_time = y_time.unsqueeze(1)

# 3. Combine into TensorDataset
# The dataset will yield tuples: (encoded_x, y_activity, y_time)
dataset = TensorDataset(encoded_x, y_activity, y_time)

# 4. Create and return DataLoader
loader = DataLoader(dataset, batch_size=batch_size, shuffle=shuffle)

return loader

```

The training loss is extremely large because the remaining_time values are in seconds and can reach up to 7,898,136 seconds. I need to normalize the remaining_time before training. Please update the "create_data_loader" function to add a normalization step: compute the mean and standard deviation of the 'remaining_time' column, normalize the values using $(\text{value} - \text{mean}) / \text{std}$, and return the mean and std values alongside the DataLoader so they can be used later for denormalization. The function signature should become: "create_data_loader(prefixes, encoded_x, vocab, batch_size=32, shuffle=True) → tuple returning "(DataLoader, float, float)" where the two floats are mean and std

🚀 LISA Flash

Here is the updated `create_data_loader` function. It now calculates the mean and standard deviation of the `remaining_time` column, normalizes the data, and returns the DataLoader along with these statistics for later use in denormalization.

python

⌵ Свернуть ⏏ Запустить 📋 Копировать

```

import pandas as pd
import torch
from torch.utils.data import TensorDataset, DataLoader
from typing import Dict, Tuple

def create_data_loader(
    prefixes: pd.DataFrame,
    encoded_x: torch.Tensor,
    vocab: Dict[str, int],
    batch_size: int = 32,
    shuffle: bool = True
) -> Tuple[DataLoader, float, float]:

```

```
"""
```

```
    Creates a DataLoader for training, including normalization of the target  
'remaining_time'.
```

```
Args:
```

```
    prefixes (pd.DataFrame): DataFrame containing 'next_activity' and  
'remaining_time' columns.  
    encoded_x (torch.Tensor): Encoded input sequences of shape (N, max_len).  
    vocab (dict): Vocabulary mapping activity names to indices.  
    batch_size (int): Batch size for the DataLoader.  
    shuffle (bool): Whether to shuffle the data.
```

```
Returns:
```

```
    Tuple[DataLoader, float, float]:  
        - DataLoader yielding batches of (x, y_activity, y_time_normalized).  
        - Mean of the original remaining_time values.  
        - Standard deviation of the original remaining_time values.
```

```
"""
```

```
# 1. Create y_activity (Next Activity Labels)  
next_activities = prefixes['next_activity'].apply(lambda x: vocab.get(x, 0))  
y_activity = torch.tensor(next_activities.values, dtype=torch.long)  
  
# 2. Prepare y_time for Normalization  
raw_times = prefixes['remaining_time'].values.astype(float)  
  
# Compute Mean and Standard Deviation  
mean_val = raw_times.mean()  
std_val = raw_times.std()  
  
# Handle case where std might be 0 (constant values) to avoid division by zero  
if std_val == 0:  
    std_val = 1.0  
  
# Normalize: (value - mean) / std  
normalized_times = (raw_times - mean_val) / std_val  
y_time = torch.tensor(normalized_times, dtype=torch.float32)  
  
# Ensure y_time has shape (N, 1)  
if y_time.dim() == 1:  
    y_time = y_time.unsqueeze(1)
```

```
# 3. Combine into TensorDataset
dataset = TensorDataset(encoded_x, y_activity, y_time)

# 4. Create and return DataLoader
loader = DataLoader(dataset, batch_size=batch_size, shuffle=shuffle)

return loader, mean_val, std_val
```

